axi_ram.v DUT 상세 분석

AXI4 Full UVM Verification 프로젝트 — DUT Study Note

1. 개요

본 문서는 AXI4 full UVM verification 프로젝트에서 DUT로 사용할 axi_ram.v 모듈에 대한 상세 분석 노트이다. 원본은 Alex Forencich의 오픈소스 프로젝트 verilog-axi에서 가져왔으며, MIT 라이센스로 배포된다.

1.1 verilog-axi 라이브러리 구조

verilog-axi repo는 AXI4 관련 building block들의 라이브러리이다. 실제 SoC 설계에서 필요한 여러 모듈이 파일 단위로 나뉘어 있고, 본 프로젝트에서는 그중 axi_ram.v 하나만 DUT로 사용한다.

주요 폴더 및 파일 카테고리:

카테고리	파일 예시	역할
RAM 계열	axi_ram.v, axi_ram_rd.v, axi_ram_wr.v	본 프로젝트 DUT (axi_ram.v)
Interconnect	axi_crossbar.v, axi_interconnect.v	여러 master/slave 라우팅 (미사용)
Adapter	axi_adapter.v, axi_axil_adapter.v	버스 폭/포맷 변환 (미사용)
Utility	axi_fifo.v, axi_register_rd.v	FIFO, pipeline 등 헬퍼 (미사용)

1.2 axi_ram.v 요약

AXI4 full 프로토콜을 지원하는 on-chip RAM slave 모듈이다. 실제 SRAM처럼 동작하며 AXI 인터페이스를 통해 burst read/write를 처리한다. Verilog 2001로 작성되어 있고, 파라미터를 통해 데이터 폭 · 주소 폭 · ID 폭을 조절 가능하다.

2. 파라미터

파라미터	기본값	의미
DATA_WIDTH	32	데이터 버스 폭 (bit)
ADDR_WIDTH	16	주소 폭 (bit) → 메모리 크기: 2^16 = 64KB
STRB_WIDTH	DATA_WIDTH/8 = 4	Write strobe 폭 (byte lane 개수)
ID_WIDTH	8	AXI ID 폭 (outstanding 구분용)
PIPELINE_OUTPUT	0	1일 때 R 채널 출력에 pipeline stage 추가 (timing 여유)

검증 관점 참고: 이 파라미터들을 다양한 값으로 sweep하여 config coverage를 잡을 수 있다.

3. 포트 — 5개 AXI 채널

AXI4 프로토콜은 5개의 독립 채널로 구성된다. Write 트랜잭션은 AW · W · B 3개 채널을, Read 트랜잭션은 AR · R 2개 채널을 사용한다. 각 채널은 서로 독립적으로 VALID/READY handshake를 통해 통신한다.

3.1 Write 관련 채널

AW 채널 (Write Address) — "어디에, 어떻게 쓸 것인가"

신호	의미
s_axi_awid	이번 transaction의 ID (여러 개 구분용)
s_axi_awaddr	시작 주소 (byte 단위)
s_axi_awlen	burst 길이 - 1 (예: len=3이면 4 beats)
s_axi_awsize	한 beat당 byte 수 (2^size)
s_axi_awburst	FIXED (00) / INCR (01) / WRAP (10)
s_axi_awvalid / awready	handshake

W 채널 (Write Data) — "실제 데이터"

신호	의미
s_axi_wdata	실제 write data
s_axi_wstrb	byte 단위 write mask (어느 byte lane을 쓸지)
s_axi_wlast	burst의 마지막 beat인지 표시
s_axi_wvalid / wready	handshake

B 채널 (Write Response) — "완료 ACK"

신호	의미
s_axi_bid	어느 transaction에 대한 응답인지
s_axi_bresp	OKAY (00) / EXOKAY / SLVERR / DECERR
s_axi_bvalid / bready	handshake

3.2 Read 관련 채널

AR 채널 (Read Address) — "어디서, 어떻게 읽을 것인가". 신호 구조는 AW 채널과 동일 (awid → arid, awaddr → araddr 등).

R 채널 (Read Data + Response) — Write와 달리 data와 response가 한 채널에 통합되어 있음

신호	의미
s_axi_rid	어느 transaction의 응답인지
s_axi_rdata	실제 read data
s_axi_rresp	응답 코드
s_axi_rlast	마지막 beat 표시
s_axi_rvalid / rready	handshake

4. 메모리 구조

4.1 배열 선언

```
reg [DATA_WIDTH-1:0] mem[(2**VALID_ADDR_WIDTH)-1:0];
```

이 선언을 풀어보면 다음과 같다:

구성 요소	의미	기본값 기준
mem	메모리 배열 이름	-
[DATA_WIDTH-1:0]	각 칸(entry)의 크기 (bit)	32비트
[(2**VALID_ADDR_WIDTH)-1:0]	칸(entry)의 총 개수	2^14 = 16,384개

따라서 총 메모리 크기는:

```
16,384 칸 × 32비트 = 16,384 × 4바이트 = 65,536 바이트 = 64 KB
```

4.2 왜 주소 변환이 필요한가

AXI에서 주소(s_axi_awaddr)는 byte 단위로 온다. 즉 주소 0은 0번 byte, 주소 4는 4번 byte를 가리킨다. 그러나 우리 메모리는 32비트(4바이트) 단위로 저장되어 있어, byte 주소를 배열 인덱스로 변환하려면 4로 나누어야 한다 (= 2비트 오른쪽 시프트).

```
byte 주소:  0  1  2  3  ¦  4  5  6  7  ¦  8  9 10 11  ¦ ...
            ^             ^             ^
            ¦             ¦             ¦
          mem[0]        mem[1]        mem[2]
```

실제 변환 코드:

```
wire [VALID_ADDR_WIDTH-1:0] s_axi_awaddr_valid =
    s_axi_awaddr >> (ADDR_WIDTH - VALID_ADDR_WIDTH);
// 기본값 기준: s_axi_awaddr >> 2
```

4.3 VALID_ADDR_WIDTH의 의미

```
parameter VALID_ADDR_WIDTH = ADDR_WIDTH - $clog2(STRB_WIDTH);
// 기본값: 16 - $clog2(4) = 16 - 2 = 14
```

AXI byte 주소는 16비트이지만, 하위 2비트는 한 word 안의 어느 byte인지를 골라주는 용도이므로 배열 인덱스로 사용되지 않는다. 즉 실제 배열 인덱스로 유효한 것은 상위 14비트뿐이다.

```
16비트 byte 주소:
  [ 상위 14비트: word 인덱스 ][하위 2비트: byte offset]
   ↓                          ↓
 mem[...]에서 몇 번째 칸?      한 칸 안의 어느 byte?
```

예시: 주소 0x0008의 접근

```
16진수 0x0008 = 2진수 0000000000_0010_00
상위 14비트: 0000000000_0010 = 2  →  mem[2]
하위 2비트:  00                →  mem[2]의 byte 0
```

4.4 STRB_WIDTH와 Byte Lane

1 byte는 항상 8 bit이다. Write strobe(WSTRB)는 byte 단위의 write mask로, 32비트 데이터를 쓸 때 어느 byte lane을 실제로 쓸지 결정한다.

```
32-bit WDATA:  [ byte3 ][ byte2 ][ byte1 ][ byte0 ]
                 8bit    8bit    8bit    8bit    <- 각 byte는 8비트
```

```
                ^       ^       ^       ^
WSTRB[3] WSTRB[2] WSTRB[1] WSTRB[0]  <- 4비트 strobe
```

예: WSTRB = 4'b0011이면 하위 2 byte만 쓰고 상위 2 byte는 유지된다.

4.5 파생 파라미터 정리

이름	기본값	의미
DATA_WIDTH	32	데이터 버스 전체 비트 수
STRB_WIDTH	4	strobe 신호 개수 = byte lane 개수
WORD_WIDTH	4	byte lane 개수 (STRB_WIDTH와 동일)
WORD_SIZE	8	byte lane 하나의 비트 수 (= 1 byte)
VALID_ADDR_WIDTH	14	배열 인덱스 유효 비트 수

혼동 주의: Alex의 코드에서 `WORD_SIZE`가 8이라는 것은 '한 byte lane이 8비트'라는 뜻이지 word 자체의 크기(32비트)가 아니다. 사실상 `bits_per_byte`에 가까운 개념이다.

4.6 실제 memory 쓰기 로직

```
for (i = 0; i < WORD_WIDTH; i = i + 1) begin
    if (mem_wr_en & s_axi_wstrb[i]) begin
        mem[write_addr_valid][WORD_SIZE*i +: WORD_SIZE]
            <= s_axi_wdata[WORD_SIZE*i +: WORD_SIZE];
    end
end
```

이 for 루프는 byte lane 4개를 하나씩 돌면서 WSTRB가 켜진 byte lane만 memory에 쓴다. `+:` 연산자는 Verilog의 part-select로, `[8*i +: 8]`은 '8*i 위치부터 8비트'를 의미한다.

5. Write 경로 — 3-state FSM

5.1 상태 정의

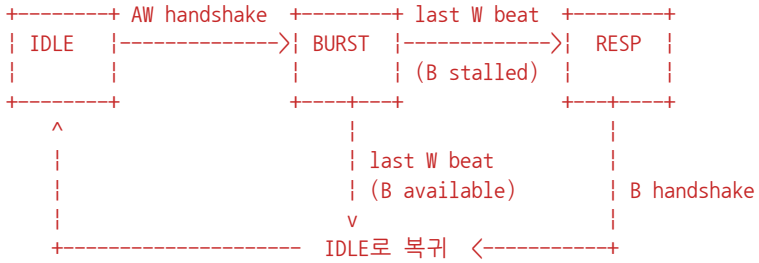
Write 경로는 3개 상태를 갖는다. FSM 상태는 localparam으로 심볼릭 상수로 정의된다.

```
localparam [1:0]
    WRITE_STATE_IDLE  = 2'd0,   // = 2'b00
    WRITE_STATE_BURST = 2'd1,   // = 2'b01
    WRITE_STATE_RESP  = 2'd2;   // = 2'b10

reg [1:0] write_state_reg = WRITE_STATE_IDLE, write_state_next;
```

> 중요: WRITE_STATE_IDLE은 함수가 아니라 상수(값)이다. FSM에서 각 상태를 숫자로 표현할 때, 그 숫자에 이름을 붙인 것이다. localparam은 C의 #define이나 const와 비슷하다.

5.2 상태 전이 다이어그램



5.3 IDLE 상태

초기 상태. AWREADY를 어썻하고 AW handshake를 기다린다.

```
WRITE_STATE_IDLE: begin
    s_axi_awready_next = 1'b1;  // AW 받을 준비
    if (s_axi_awready && s_axi_awvalid) begin
        // AW handshake 성공! 정보 저장
        write_id_next    = s_axi_awid;
        write_addr_next  = s_axi_awaddr;
        write_count_next = s_axi_awlen;
        write_size_next  = s_axi_awsize;
        write_burst_next = s_axi_awburst;

        s_axi_awready_next = 1'b0;  // 다음 AW는 안 받음
        s_axi_wready_next  = 1'b1;  // W 받을 준비
        write_state_next   = WRITE_STATE_BURST;
    end
end
```

핵심: AW 정보를 레지스터에 저장해두고 이후 상태에서 사용한다.

5.4 BURST 상태

W data를 받으면서 memory에 쓴다.

```
WRITE_STATE_BURST: begin
    s_axi_wready_next = 1'b1;
    if (s_axi_wready && s_axi_wvalid) begin
        mem_wr_en = 1'b1;  // 이번 clock에 memory write

        // INCR/WRAP이면 주소 증가 (FIXED는 그대로)
        if (write_burst_reg != 2'b00) begin
            write_addr_next = write_addr_reg + (1 << write_size_reg);
        end
```

```
        write_count_next = write_count_reg - 1;

        if (write_count_reg > 0) begin
            write_state_next = WRITE_STATE_BURST;  // 계속
        end else begin
            // 마지막 beat!
            s_axi_wready_next = 1'b0;
            if (s_axi_bready || !s_axi_bvalid) begin
                s_axi_bid_next     = write_id_reg;
                s_axi_bvalid_next  = 1'b1;
                s_axi_awready_next = 1'b1;
                write_state_next   = WRITE_STATE_IDLE;
            end else begin
                write_state_next = WRITE_STATE_RESP;
            end
        end
    end
end
```

5.5 RESP 상태

BREADY가 늦게 오는 경우 진입. B handshake가 완료되면 IDLE로 복귀한다.

```
WRITE_STATE_RESP: begin
    if (s_axi_bready || !s_axi_bvalid) begin
        s_axi_bid_next     = write_id_reg;
        s_axi_bvalid_next  = 1'b1;
        s_axi_awready_next = 1'b1;
        write_state_next   = WRITE_STATE_IDLE;
    end
end
```

5.6 Write 타이밍 예시

AWLEN=3 (4 beats), AWADDR=0x40, INCR, size=2 (4바이트/beat) 시나리오:

```
Clock:      1     2     3     4     5     6     7     8
AWVALID: ---+-----+-----+_____|_____|_____|_____|_____|
AWREADY: ---+-----+_____|_____|_____|_____|_____|_____|
AWADDR:     0x40
AWLEN:      0x03

WVALID:             +-----+-----+-----+-----+_____|_____|
WREADY:             +-----+-----+-----+-----+_____|_____|
WDATA:              D0    D1    D2    D3
WLAST:              0     0     0     1

State:    IDLE  IDLE  BURST BURST BURST BURST BURST IDLE
mem_wr_en:              1     1     1     1
mem[16]:            <-  D0                          (0x40>>2 = 16)
mem[17]:                  <-  D1
mem[18]:                        <-  D2
mem[19]:                              <-  D3

BVALID:                               +-----+___
BREADY:                               +-----+___
```

6. Read 경로 — 2-state FSM

6.1 상태 정의

Read 경로는 Write보다 단순한 이유는 response 채널이 별도로 없고 R 채널이 data와 response를 겸하기 때문이다.

```
localparam [0:0]
    READ_STATE_IDLE  = 1'd0,
    READ_STATE_BURST = 1'd1;

reg [0:0] read_state_reg = READ_STATE_IDLE, read_state_next;
```

6.2 상태 전이 다이어그램

```
    +--------+ AR handshake  +--------+
    ¦ IDLE   ¦-------------->¦ BURST  ¦
    ¦        ¦              ¦        ¦
    +--------+              +----+---+
        ^                        ¦
        ¦     마지막 R handshake  ¦
        +-------------------------+
```

6.3 IDLE 상태

```
READ_STATE_IDLE: begin
    s_axi_arready_next = 1'b1;
    if (s_axi_arready && s_axi_arvalid) begin
        read_id_next    = s_axi_arid;
        read_addr_next  = s_axi_araddr;
        read_count_next = s_axi_arlen;
        read_size_next  = s_axi_arsize;
        read_burst_next = s_axi_arburst;

        s_axi_arready_next = 1'b0;
        read_state_next    = READ_STATE_BURST;
    end
end
```

6.4 BURST 상태

memory에서 읽어서 R 채널로 송출.

```
READ_STATE_BURST: begin
    if (s_axi_rready || (PIPELINE_OUTPUT && !s_axi_rvalid_pipe_reg)
                     || !s_axi_rvalid_reg) begin
        mem_rd_en         = 1'b1;
        s_axi_rvalid_next = 1'b1;
        s_axi_rid_next    = read_id_reg;
        s_axi_rlast_next  = (read_count_reg == 0);

        if (read_burst_reg != 2'b00) begin
            read_addr_next = read_addr_reg + (1 << read_size_reg);
        end
        read_count_next = read_count_reg - 1;

        if (read_count_reg > 0) begin
            read_state_next = READ_STATE_BURST;
        end else begin
            s_axi_arready_next = 1'b1;
            read_state_next    = READ_STATE_IDLE;
        end
    end
```

```
end
```

실제 memory read는 clocked always block에서 수행:

```
if (mem_rd_en) begin
    s_axi_rdata_reg <= mem[read_addr_valid];
end
```

6.5 Read 타이밍 예시

ARLEN=3 (4 beats), ARADDR=0x40, INCR, size=2:

```
Clock:      1     2     3     4     5     6     7
ARVALID: ---+-----+_____|_____|_____|_____|_____|
ARREADY: ---+-----+_____|_____|_____|_____|_____|
ARADDR:     0x40
ARLEN:      0x03

State:    IDLE  IDLE  BURST BURST BURST BURST IDLE
mem_rd_en:              1     1     1     1

RVALID:                 +-----+-----+-----+-----+___
RREADY:                 +-----+-----+-----+-----+___
RDATA:                  D0    D1    D2    D3
RLAST:                  0     0     0     1
RID:                    ID    ID    ID    ID
```

6.6 Write vs Read 요약 비교

측면	Write	Read
채널 수	3개 (AW, W, B)	2개 (AR, R)
FSM 상태	3개 (IDLE, BURST, RESP)	2개 (IDLE, BURST)
데이터 흐름	Master → Slave (W)	Slave → Master (R)
응답 방식	별도 B 채널, burst 끝난 후 한 번	R 채널에 매 beat마다 붙어서
RESP 상태 존재 이유	B 채널 별도 → backpressure 가능	R 채널이 data와 통합 → 별도 상태 불필요

7. DUT 제약사항 (검증 관점)

이 DUT는 AXI4 spec을 완전히 준수하지 않는다. 검증 계획을 세울 때 반드시 인지해야 하는 제약사항들을 아래에 정리한다. 이 제약사항들은 verification 어필 포인트가 될 수 있다.

7.1 WRAP Burst 처리 오류

AXI spec에서 `burst[1:0]` 인코딩:

값	타입	동작
00	FIXED	주소 그대로 유지
01	INCR	매 beat마다 주소 증가
10	WRAP	wrap boundary에서 wrap-around
11	Reserved	-

```
// 이 DUT의 처리 로직:
if (write_burst_reg != 2'b00) begin
    write_addr_next = write_addr_reg + (1 << write_size_reg);
end
```

문제: 이 DUT는 `burst != 2'b00`이면 그냥 주소를 증가시킨다. 즉 WRAP burst일 때도 INCR처럼 처리하며, wrap boundary에서의 wrap-around 로직이 없다.

7.2 Outstanding Transaction 미지원

Write와 Read 각각의 FSM이 IDLE → BURST → (RESP) → IDLE로 완결되는 구조라, 이전 트랜잭션이 완료되기 전까지는 다음 AW/AR을 수용하지 않는다.

방향	Outstanding 최대	비고
Write (AW)	1	IDLE → BURST → RESP 완료 후 다음 AW 수용
Read (AR)	1	IDLE → BURST 완료 후 다음 AR 수용
Read + Write	병렬 처리 가능	두 FSM 독립, 채널 독립성 유지

7.3 하드코딩된 응답 (RESP)

```
assign s_axi_bresp = 2'b00;   // 항상 OKAY
assign s_axi_rresp = 2'b00;   // 항상 OKAY
```

SLVERR/DECERR 응답이 발생하지 않는다. Address wrap-around가 자동으로 발생해서 존재하지 않는 주소 접근 시에도 OKAY를 리턴한다. Exclusive access (`awlock`) 무시되어 EXOKAY도 없다.

7.4 무시되는 신호

```
awcache, awprot, awlock  // 입력받지만 사용 안 함
arcache, arprot, arlock  // 입력받지만 사용 안 함
```

Cacheable/protection attribute는 그냥 통과된다. 검증 시 이 신호들에 random 값을 주고 'data에 영향 없음'만 확인 가능하다.

8. 검증 어필 포인트 (이력서용)

8.1 검증 계획 조정 사항

Week	기존 계획	이 DUT 반영 조정
4 (Scoreboard)	INCR/WRAP burst 검증	WRAP burst에 대해 spec대로 정확한 reference model 구현 → DUT와 mis
5 (Sequences)	Multi-outstanding sequence	축소 (어차피 이 DUT는 back-pressure). Read/Write 동시 실행 sequence 추
6 (SVA/Error)	BRESP/RRESP 응답 검증	스킵 (항상 OKAY). Protocol timing SVA에 집중

8.2 예상 'Bugs Detected' 리스트

검증을 통해 발견할 수 있는 spec 위반 사항들:

1. WRAP burst treated as INCR — wrap boundary 계산 로직 없음 (spec violation)
2. No SLVERR on out-of-range address — 조용히 wraparound
3. Exclusive access silently ignored — EXOKAY 응답 없음
4. Cacheable/protection attributes ignored — awprot, awcache 사용 안 됨

8.3 이력서 문장 초안

AXI4 Full UVM Verification of Open-Source RAM Slave — Jul 2026

• Architected a full UVM environment for the axi_ram open-source AXI4 slave (alexforencich/verilog-axi); implemented cycle-accurate reference model with INCR/WRAP burst address calculation and byte-lane strobe tracking.

• Identified 4 specification deviations in the DUT via scoreboard mismatch analysis, including WRAP burst treated as INCR (missing wrap boundary logic) and silent OKAY response on out-of-range accesses; reported findings via upstream GitHub issue.

• Achieved 100% functional coverage across burst_type × length × size × ID cross-buckets and 20+ SVA protocol properties, validated via automated Questa regression with multi-seed coverage merging.

9. 부록 — FSM 문법 개념

9.1 localparam으로 상태에 이름 붙이기

`WRITE_STATE_IDLE` 같은 것들은 함수가 아니라 상수이다. FSM에서 각 상태를 숫자로 표현할 때 그 숫자에 이름을 붙인 것으로, 이걸 symbolic constant 또는 enumerated encoding이라고 부른다.

숫자만 쓰면 가독성이 나쁘다:

```
case (write_state_reg)
    2'd0: begin  // 이게 뭔 상태인지 알기 어려움
        ...
    end
    2'd1: begin
        ...
    end
endcase
```

이름을 붙이면 훨씬 명확하다:

```
case (write_state_reg)
    WRITE_STATE_IDLE: begin   // 아! IDLE 상태
        ...
```

```
    end
    WRITE_STATE_BURST: begin  // BURST 상태
        ...
    end
endcase
```

9.2 하드웨어로 무엇이 만들어지나

합성 시에 만들어지는 하드웨어:

- `write_state_reg` → 2비트짜리 flip-flop (값 0, 1, 2 저장)
- case 문 → flip-flop 값을 보고 로직을 선택하는 멀티플렉서
- 함수 호출 개념은 없음 → 조합회로 + 순차회로

9.3 FSM 3요소

요소	역할	이 DUT에서
State register	현재 상태 저장	write_state_reg, read_state_reg
Next state logic	다음 상태 결정	always @* 블록의 case 문
Output logic	상태에 따른 출력 생성	각 case의 신호 assignment

— 문서 끝 —